

Implement Client Script & UI Policy (Incident)

Complete technical specification, step-by-step implementation, scripts, and multi-scenario verification guide.

1. Overview & Objectives

Problem Statement: Incident records frequently require consistent and accurate data entry to facilitate effective triage, routing, and resolution. Relying solely on user awareness and manual checks often results in incomplete, inconsistent, or incorrect data. This directly degrades reporting accuracy, SLA compliance, and overall service quality. Enforcing conditional field behavior and validation directly at the user interface tier resolves these discrepancies before submission.

Objective: Demonstrate how ServiceNow client-side controls dynamically enforce data integrity on Incident records. By combining UI Policies, UI Policy Actions, and Client Scripts (onChange, onSubmit, onCellEdit), this solution automates field assignments, controls mandatory/read-only states, and blocks unauthorized submissions both on forms and list views.

Core Skills Covered

Incident
Management

UI Policies &
Actions

onChange Client
Scripts

onSubmit
Validation

onCellEdit List
Blocking

2. Task 1: Create UI Policy on Incident

Create a UI Policy to manage conditional UI behavior dynamically based on incident impact.

1. Log in to the ServiceNow instance with administrative or configuration privileges.
2. Navigate to: System UI → UI Policies.
3. Click **New** to open a new UI Policy record.
4. Fill in the UI Policy configuration parameters:

Configuration Field	Value	Description / Purpose
Short description (Name)	High Impact Control	Identifies the policy's purpose to govern high impact incident behavior.
Table	Incident [incident]	Specifies that the UI Policy is scoped to the Incident table.
Active	true (Checked)	Enables the policy so it is actively enforced on incident forms.
Global	true (Checked)	Applies policy across all form views.
On load	true (Checked)	Executes evaluation when the form loads and when values change.
Reverse if false	true (Checked)	Automatically reverts applied actions whenever conditions evaluate to false.

1. Configure the **When to Apply / Conditions** filter:
 - **Field:** Impact
 - **Operator:** is
 - **Value:** 1 - High
2. In the **UI Policy Actions** related list, add an action for **Assignment group**:
 - **Field name:** Assignment group
 - **Mandatory:** True
 - **Read only:** False (or Leave alone)
3. Click **Submit / Update** to save the UI Policy.

3. Task 2: Create UI Policy Action - Urgency

Configure a UI Policy Action under the *High Impact Control* policy to lock the Urgency field when Impact is High.

1. Navigate back to the newly created **High Impact Control** UI Policy record.
2. Scroll down to the **UI Policy Actions** related list and click **New**.
3. Configure the action details:

Field	Value	Behavioral Impact
Field name	Urgency	Targets the Urgency field on the Incident form.
Read only	True	Locks the Urgency field from manual edits whenever Impact is High.
Mandatory	Leave alone	Does not modify mandatory requirements.
Visible	Leave alone	Ensures Urgency remains visible on the form interface.

1. Click **Submit** to attach this action to the *High Impact Control* UI Policy.

4. Task 3: Create onChange Client Script

Create an onChange Client Script to automate the Urgency setting whenever an agent selects a High Impact value.

1. Navigate to: System UI → Client Scripts.
2. Click **New**.
3. Configure the client script header:
 - **Name:** Auto set urgency for high impact
 - **Table:** Incident [incident]
 - **UI Type:** Desktop (or All)
 - **Type:** onChange
 - **Field name:** Impact
 - **Active:** true
 - **Global:** true
4. Enter the following script in the Script editor:

```
function onChange(control, oldValue, newValue, isLoading) {  
    if (isLoading || newValue == '') {  
        return;  
    }  
  
    // Check if Impact is set to 1 - High  
    if (newValue == '1') {  
        g_form.setValue('urgency', '1');  
        g_form.addInfoMessage('High impact incidents must have high urgency and an assignment group.');    }  
}
```

1. Click **Submit** to save and activate the script.

5. Task 4: Create onSubmit Client Script (Save Validation)

Enforce strict save-time validation preventing submission of high impact incidents without an assigned owner.

1. Navigate to: System UI → Client Scripts and click **New**.
2. Configure script parameters:
 - **Name:** Prevent save if assigned to missing
 - **Table:** Incident [incident]
 - **UI Type:** Desktop (or All)
 - **Type:** onSubmit
 - **Active:** true
 - **Global:** true
3. Enter the validation logic in the script editor:

```
function onSubmit() {
    // If Impact is High ('1') and Assigned To is empty, abort submission
    if (g_form.getValue('impact') == '1' && g_form.getValue('assigned_to') == '') {
        g_form.showErrorMessage(
            'assigned_to',
            'Assigned To is mandatory for High impact incidents.'
        );
        return false; // Prevents form submission
    }
    return true; // Allows form submission
}
```

1. Click **Submit** to persist the script.

6. Task 5: Create onCellEdit Client Script (List Edit Blocking)

Prevent users from bypassing lifecycle stages and approval policies by modifying Incident State directly in list views.

1. Navigate to: System UI → Client Scripts and click **New**.
2. Configure the client script header:
 - **Name:** Prevent state change via list edit
 - **Table:** Incident [incident]
 - **UI Type:** Desktop (or All)
 - **Type:** onCellEdit
 - **Field name:** State
 - **Active:** true
3. Enter the callback rejection script:

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    // Alert the user that list modifications for State are disallowed
    alert('State cannot be changed from the list. Please open the Incident.');
```



```
    // Pass false to callback to cancel the cell update
    callback(false);
}
```

1. Click **Submit** to save the configuration.

7. Task 6: Testing & Quality Assurance Verification

Validate the client-side behaviors across all defined edge cases and scenarios.

Test Activity	Action Performed	Expected Outcome	Status
Activity 1: Mandatory Enforcement	Open Incident → Create New. Select Impact = 1 - High . Leave Assigned to blank and click Submit .	Record submission blocked. Error box displayed on Assigned to: "Assigned To is mandatory for High impact incidents."	✓ Passed
	Populate Assigned to with an active user (e.g. <i>Beth Anglin</i>) with	Record saves successfully. Priority calculates to 1 - Critical,	✓ Passed

Test Activity	Action Performed	Expected Outcome	Status
Activity 2: Successful Submission	Impact = 1 - High. Click Submit .	Urgency is read-only (1 - High), info message visible.	
Activity 3: Reverse Condition Test	Open an existing High Impact incident. Change Impact from 1 - High to 2 - Medium.	Urgency unlocks and becomes editable. Mandatory requirement on Assigned to is lifted. Saves cleanly.	✓ Passed
Activity 4: List Edit Blocking	Navigate to Incident → All. Double-click the State cell in the list to attempt an inline change.	Browser popup: <i>"State cannot be changed from the list. Please open the Incident."</i> Cell reverts immediately.	✓ Passed
Activity 5: Form-Based Update	Open the Incident form directly. Modify State (e.g., to In Progress) and click Update .	State update succeeds without restriction, proving list editing is blocked while legitimate form edits are preserved.	✓ Passed

8. Conclusion & Architectural Best Practices

The **Implement Client Script & UI Policy (Incident)** implementation successfully demonstrates the coordinated application of client-side mechanisms within ServiceNow:

- **UI Policies vs. Client Scripts:** UI Policies provide lightweight, codeless field property management (Mandatory, Read-Only, Visible) that cleanly self-revert using *Reverse if false*. Client scripts provide programmatic event handling (onChange, onSubmit, onCellEdit) for dynamic value assignment, modal user messaging, and conditional save validation.
- **Multi-Channel Integrity:** Combining onSubmit with onCellEdit ensures business rules cannot be bypassed either through hasty form submission or through list inline editing.
- **Operational Efficiency:** This layered client-side design drastically cuts down administrative triage overhead, eliminates invalid records at the point of capture, and enforces immediate platform data quality compliance.